

AI CALLER AGENT

Sprint Planning Guide

Week-by-week execution calendar for V1

Version 1.1

13-week V1 timeline, anchored to the Team Execution Plan's 9 integration checkpoints

Companion to the Team Execution Plan -- ticket IDs referenced here are defined there in full

Table of Contents

1. What Is a Sprint, and Why Plan This Way?

3

1.1 Why agile development uses sprints at all

3

1.2 Why parallel work matters here specifically

3

Week 1 -- Foundation

4

Week 2 -- Voice Skeleton

5

Week 3 -- Real Speech In, Real Speech Out

5

Week 4 -- Turns, TTS, and Persistence Meet

6

Week 5 -- The Agent Gets a Brain: RAG

7

Week 6 -- Multi-Tenancy, Part 1 -- Auth & RBAC

7

Week 7 -- Multi-Tenancy, Part 2 -- Tenant Context, RLS & Isolation Proof

8

Week 8 -- Multilingual

9

Week 9 -- Dashboards, Part 1

9

Week 10 -- Dashboards, Part 2

10

Week 11 -- Automations & Billing

10

Week 12 -- Production Hardening

11

Week 13 -- Pilot Readiness

12

1. What Is a Sprint, and Why Plan This Way?

CONCEPT · Sprint

WHAT IT IS

A sprint is a fixed, short time period -- here, one week -- with a specific goal, at the end of which the team checks: did we actually get there, and can we show working software, not just a list of things attempted.

REAL-WORLD ANALOGY

It's the difference between a road trip with no map ("we'll get there eventually") and one broken into named daily legs, each ending at a real town where you can check the fuel gauge and the map before deciding the next leg. A sprint is one leg -- short enough that if something's wrong, you find out in days, not months.

WHY STAYKARO USES IT

Three people are building interdependent pieces (Execution Plan §G, the dependency graph). A week-long sprint with a named goal and a demo checklist is what turns "Nishkala is working on auth" into something the other two can actually plan around -- they know what should exist by when.

TECHNICAL IMPLEMENTATION

Each week below has a goal, the specific tickets (by ID, from the Team Execution Plan) each person works, expected output, dependencies, risks, a demo checklist, and acceptance criteria -- the same shape every week, so the format itself becomes familiar fast.

1.1 Why agile development uses sprints at all

The alternative -- plan the whole project in detail up front, then execute for three months without checking in -- fails specifically when requirements or reality shift partway through, which is normal, not exceptional. A sprint boundary is a cheap, regular checkpoint: if Week 4's multilingual retrieval testing (NK-11) turns up a problem with the chosen embedding model, that's a one-week surprise to absorb and replan around, not a three-month one discovered at the end.

1.2 Why parallel work matters here specifically

With three people and twelve phases, working strictly one phase at a time end-to-end would take far longer than necessary -- most phases don't actually need all three people simultaneously. The Team Execution Plan's Parallel Development Timeline (§F) and Dependency Graph (§G) already identify exactly which pieces can run at the same time; this document turns that into an actual week-by-week calendar, anchored to the same nine integration checkpoints.

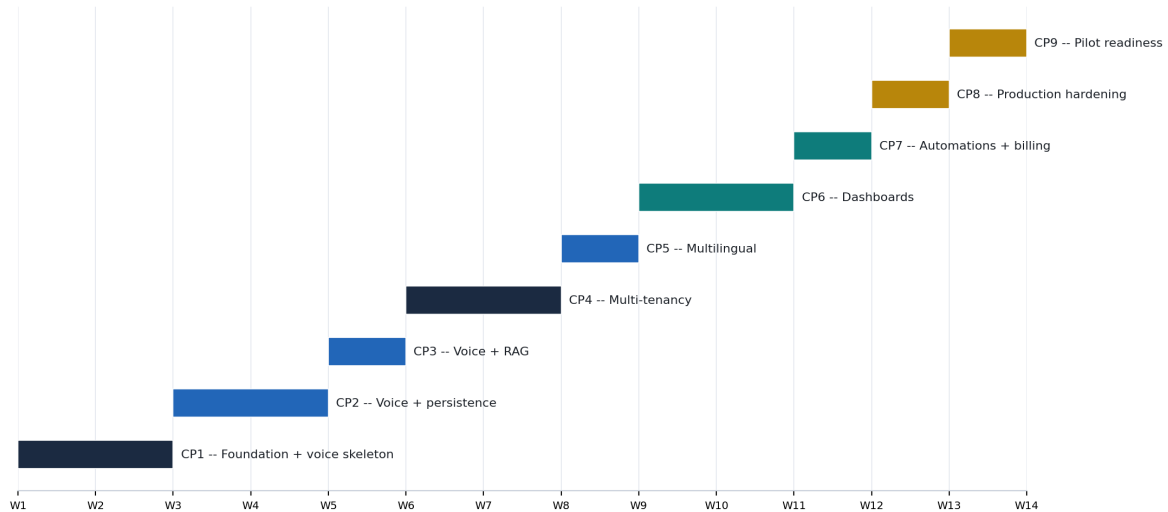


Figure 1.1 -- Thirteen-week V1 timeline by integration checkpoint (CP1-CP9). Checkpoints overlap deliberately -- see each week's detail for exactly who is doing what.

A NOTE ON DATES

Week numbers are relative to project start, not calendar dates -- insert actual start date once Phase 0 begins. Durations assume no major blocking surprises; treat Weeks 6-7 (multi-tenancy) as the single most schedule-risky pair of weeks in the plan -- see Week 6's Risks.

Week 1 -- Foundation

Goal: All base services start reliably through Docker; the team can build on a shared, working local environment.

Shivashree	Nishkala	Nihal
Scratch Pipecat/Deepgram/Groq/Cartesia pipeline (no dependency on NK/NH); draft provider interface shapes; provision Exotel/Twilio trial accounts.	NK-01 FastAPI skeleton, NK-02 Postgres schema + migrations, NK-03 Redis session service, NK-04 health checks.	NH-01 Docker Compose skeleton, NH-02 Dockerfiles, NH-03 Nginx config, NH-04 CI/CD pipeline, NH-05 VPS deploy scripting, NH-16 (start) n8n licensing email sent.

Expected output

- docker compose up brings every service up healthy on a clean machine
- Empty-shell API responds to /health and /health/ready
- n8n licensing question formally sent -- longest external lead time in the plan, started day one

Dependencies

- None -- this is the starting week

Risks

- Environment drift between three laptops if Docker Compose isn't the very first thing everyone runs -- mandate it day one, not "when convenient"
- n8n licensing reply may take longer than a week -- track it, don't let it go quiet

Demo checklist

- docker compose up on a clean checkout, live, in front of the team
- Hit /health and /health/ready and show a green response

Acceptance criteria

- Checkpoint 1 exit criteria (Execution Plan \$I): all services start via Docker; empty-call lifecycle test passes once SH-03 lands next week

Week 2 -- Voice Skeleton

Goal: A test call connects end-to-end through the pipeline shell -- proving the wiring, not yet the intelligence.

Shivashree	Nishkala	Nihal
SH-03 Pipecat pipeline skeleton + call session manager, using Week 1's scratch pipeline as a base.	Supports Shivashree's integration needs against NK-01/03; begins NK-09 ingestion pipeline groundwork.	Finishes infra hardening (NH-01-05); reviews Phase 1 voice work as it lands.

Expected output

- A call connects, holds open, and disconnects cleanly with correct Redis session lifecycle (Checkpoint 1 exit criteria)

Dependencies

- SH-03 depends on NK-01 and NK-03 from Week 1 -- confirm both merged before Monday

Risks

- If NK-03's Redis session schema changes after SH-03 starts building against it, that's a costly late rework -- freeze the schema by Monday of this week

Demo checklist

- Place a real test call, show the Redis session key appear and disappear on connect/disconnect

Acceptance criteria

- Checkpoint 1 fully green: Docker, health checks, AND empty-call lifecycle all passing

Week 3 -- Real Speech In, Real Speech Out

Goal: A caller can speak to the AI and hear a generated response -- the first time the system is actually a voice agent, even without RAG grounding yet.

Shivashree	Nishkala	Nihal
SH-01 Exotel adapter, SH-02 Twilio adapter, SH-04 Deepgram STT, SH-06 Groq LLM streaming.	NK-12 call persistence groundwork (waits on SH-11 events, starts schema-side).	Early scaffolding for NH-06 admin client screens (non-blocking prep, real build starts Week 9).

Expected output

- A real phone call to a real number gets a spoken AI response, with no RAG grounding yet (generic prompt only)

Dependencies

- SH-04/SH-06 both depend on SH-03 from Week 2

Risks

- This is the first time real telephony providers are in the loop -- budget extra time for Exotel/Twilio sandbox quirks that don't show up in local testing

Demo checklist

- Call the real trial number, have a live (ungrounded) conversation with the AI in front of the team

Acceptance criteria

- Caller can complete a full round-trip conversation turn; STT and LLM latency individually visible in logs (early SH-13 groundwork)

Week 4 -- Turns, TTS, and Persistence Meet

Goal: Every turn of a real call is durably recorded, and the AI's voice is genuinely streamed, not just generated then played.

Shivashree	Nishkala	Nihal
SH-08 TTS streaming, SH-11 turn management + turn_id/language_code generation.	NK-12 call persistence (consumes SH-11's events -- this is the week these two connect for real).	Continues dashboard scaffolding; starts drafting the API contracts (Execution Plan §J) with Nishkala.

Expected output

- A completed test call produces a correct, ordered call_turns history in PostgreSQL, matching what was actually said

Dependencies

- NK-12 blocked on SH-11 landing early in the week -- sequence accordingly, don't start both simultaneously assuming they'll meet in the middle

Risks

- Turn/message data model mismatch between what Shivashree emits and what Nishkala expects to persist -- this is exactly what Checkpoint 2 (Execution Plan §I) exists to catch; don't skip its transcript-correctness check

Demo checklist

- Play back a real call's transcript straight from the database, matching what was actually said, in order

Acceptance criteria

- Checkpoint 2 (voice + persistence) fully green

Week 5 -- The Agent Gets a Brain: RAG

Goal: The AI answers questions using a real tenant's uploaded knowledge, and correctly declines when it doesn't know.

Shivashree	Nishkala	Nihal
SH-14 voice-side RAG query integration.	NK-09 knowledge ingestion pipeline, NK-10 pgvector retrieval + tenant filter.	Early groundwork for NH-10 knowledge base UI (depends on NK-09's API shape).

Expected output

- A test call answers a scripted factual question correctly from an uploaded test document, and says it doesn't know rather than guessing on an out-of-scope question

Dependencies

- SH-14 depends on NK-10; NK-10 depends on NK-09 -- sequence within Nishkala's week matters

Risks

- This is the first tenant-isolation-relevant code (retrieval must never cross tenants) landing before Phase 4's dedicated isolation testing exists yet -- treat NK-10's own tests as a down payment on Checkpoint 4, not a substitute for it

Demo checklist

- Ask the agent a real question answerable only from the uploaded document, then ask one it has no information about

Acceptance criteria

- Checkpoint 3 (voice + RAG) fully green

Week 6 -- Multi-Tenancy, Part 1 -- Auth & RBAC

Goal: Real login and role-based access exist; nothing tenant-scoped ships past this pair of weeks without them.

Shivashree	Nishkala	Nihal
Reviewer role this week; continues SH-05 (endpoint tuning), SH-07 (OpenAI fallback), SH-09 (TTS fallback) in parallel -- Phase 2 production-voice work runs alongside Nishkala's Phase 4 work, not after it.	NK-05 authentication (JWT), NK-06 RBAC implementation.	Supports; begins real NH-06/NH-09 dashboard implementation now that auth exists to build against.

Expected output

- Login issues a valid JWT; each of the five roles reaches exactly its intended set of endpoints, verified by the RBAC matrix test

Dependencies

- NH-06/NH-09 depend on NK-05/NK-06 landing early in the week

Risks

- **This is the single highest schedule-risk pair of weeks in the whole plan.** NK-07 (tenant context/RLS, Week 7) is AI-LOW and the plan's highest-stakes review item -- don't compress it to make up time lost elsewhere; slipping a few days here is far cheaper than a cross-tenant bug reaching Phase 6+

Demo checklist

- Log in as each of the five roles, show each one blocked from an endpoint it shouldn't reach

Acceptance criteria

- RBAC matrix test (every role × every sensitive endpoint) passing in CI

Week 7 -- Multi-Tenancy, Part 2 -- Tenant Context, RLS & Isolation Proof

Goal: Cross-tenant data access is provably impossible, not just improbable -- the hard gate before anything tenant-scoped ships further.

Shivashree	Nishkala	Nihal
Serious, non-rubber-stamp review of NK-07 (this is not a formality this week); continues Phase 2/5 voice work in parallel.	NK-07 tenant context propagation + RLS policies (AI-LOW -- Nishkala's own focused implementation time, not delegated), NK-08 tenant isolation automated test suite.	Supports; continues real dashboard build (NH-06, NH-09) now against RLS-protected endpoints.

Expected output

- A connection borrowed, returned, and re-borrowed under concurrent load from different tenants' requests never carries stale tenant context (Database Design §6)
- Full cross-tenant penetration suite (NK-08) passing across API, RAG, documents, recordings, analytics, billing, exports

Dependencies

- NK-08 depends on NK-07 landing first; do not run the isolation suite against unfinished RLS work and call it passing

Risks

- **Do not compress this week.** If NK-07 isn't genuinely ready, the honest move is to hold Checkpoint 4 open rather than let Phase 6-9 start building against an unproven isolation layer -- see Execution Plan §Q

Demo checklist

- Run the isolation suite live, showing a deliberate cross-tenant read attempt fail

Acceptance criteria

- Checkpoint 4 fully green -- this is a hard gate, not a soft target

Week 8 -- Multilingual

Goal: The approved language test suite passes: English, Hindi, Telugu, and code-mixed variants, including mid-call switching.

Shivashree	Nishkala	Nihal
SH-12 turn-level language detection, SH-15 multilingual/code-mixed voice test execution.	NK-11 multilingual embedding evaluation + retrieval test suite (Database Design §16's test matrix).	Reviewer/support; signs off on SH-15's results alongside Nishkala.

Expected output

- Full §34 voice test matrix and §16 retrieval matrix both passing, including a scripted Telugu→English→Telugu call tracked correctly turn by turn

Dependencies

- SH-12 depends on SH-04 (Week 3) and SH-11 (Week 4); NK-11 depends on NK-10 (Week 5)

Risks

- Embedding model evaluation (NK-11) is AI-LOW and can surface a genuine "the current model doesn't handle Telugu retrieval well enough" finding -- budget this week with room to actually act on that, not just discover it

Demo checklist

- Live scripted call switching languages mid-conversation, agent responding correctly grounded in the right language's context each time

Acceptance criteria

- Checkpoint 5 fully green

Week 9 -- Dashboards, Part 1

Goal: Staykaro operations can see and manage clients without touching the database directly.

Shivashree	Nishkala	Nihal
Reviews NH-07 (his own instrumentation, displayed) as it's built.	Supports dashboard API needs as Nihal builds against them.	NH-06 admin client management screens, NH-07 admin live-call/system-health screens.

Expected output

- Admin can view any client's configuration and see a live call's per-stage latency breakdown in near-real-time

Dependencies

- NH-06/07 depend on Checkpoint 4 (Week 7) being genuinely green -- these are the first screens built against RLS-protected, multi-tenant endpoints

Risks

- The live-call view is the debugging tool the team will actually rely on going forward -- treat gaps here as a cost paid repeatedly later, not a one-time shortcut

Demo checklist

- Pull up a live test call on the admin dashboard and narrate its latency breakdown in real time

Acceptance criteria

- Phase 6 exit criteria: Staykaro operations can manage pilot clients without direct DB changes

Week 10 -- Dashboards, Part 2

Goal: Clients can self-manage their own agent, knowledge base, calls, and billing view -- without Staykaro engineering involvement.

Shivashree	Nishkala	Nihal
Reviewer.	Supports.	NH-08 admin usage/cost screens, NH-09 client agent-config UI, NH-10 client knowledge-base UI, NH-11 client calls/transcripts UI, NH-12 client billing UI.

Expected output

- A client can update their agent's prompt and see it take effect on the next call; upload a document and watch it reach "ready"; view only their own calls and invoices

Dependencies

- All of this depends on the corresponding backend APIs (NK-05, NK-06, NK-09, NK-12) already existing from earlier weeks

Risks

- Six screens in one week is dense -- this is exactly where Nihal's AI-HIGH ticket weighting (Execution Plan §R) is meant to carry load; if it's not landing at this pace, that's a signal worth raising early in the week, not at Friday's demo

Demo checklist

- A client-role login walks through updating the agent, uploading a document, and viewing calls/billing -- all correctly scoped to that one tenant

Acceptance criteria

- Phase 7 exit criteria: clients can self-manage allowed configuration without engineering intervention; tenant-isolation checks passing on every new client-facing screen

Week 11 -- Automations & Billing

Goal: Usage reconciles against real provider invoices, and a completed call reliably triggers post-call automation without any risk to the live call.

Shivashree	Nishkala	Nihal
Reviewer.	NK-13 usage metering service, NK-14 Razorpay integration, NK-15 webhook verification + idempotency.	NH-13 integration service skeleton, NH-14 email/WhatsApp automation, NH-15 CRM integration framework, NH-16 n8n licensing decision finalized (should have an answer by now from Week 1's early start).

Expected output

- A test call's usage record reconciles against its actual provider cost; a completed call triggers exactly one correctly formatted automation message; a duplicated Razorpay webhook does not double-charge

Dependencies

- NH-13 depends on SH-16 (call-completion event, built in an earlier week alongside SH-11)

Risks

- This is where a licensing surprise on n8n would be most expensive if it hadn't been started in Week 1 -- confirm the decision is genuinely closed, not still pending, before building production automations on an assumption either way

Demo checklist

- Kill the integration service mid-call live, show the call is completely unaffected; then show the post-call WhatsApp/email/CRM update actually arrive

Acceptance criteria

- Checkpoint 7 fully green

Week 12 -- Production Hardening

Goal: The system proves it can survive load, failure, a bad deploy, and a lost database -- not just that it works when everything goes right.

Shivashree	Nishkala	Nihal
SH-17 concurrent-call load testing support, voice reliability/performance hardening.	NK-16 audit logging, NK-17 backup strategy + actual restoration test, backend/security hardening.	NH-17 deployment draining implementation, NH-18 monitoring/alerting + load-test infrastructure.

Expected output

- A measured (not assumed) concurrent-call capacity number; a successfully restored backup with verified data integrity; a deploy triggered mid-call that does not disconnect the caller; alerts that actually fire under simulated failure

Dependencies

- This week depends on essentially everything before it -- it is deliberately scheduled last among the "build" weeks for that reason

Risks

- Four separate hard tests in one week (load, restore, drain, alerts) is ambitious -- if any single one needs more room, take it; shipping with an unproven backup or an unproven drain sequence is a worse outcome than a one-week schedule slip

Demo checklist

- Live: trigger a deploy during an active call and show it survive; then show a restored backup's data checked against the original

Acceptance criteria

- Checkpoint 8 fully green -- all four hard tests passing

Week 13 -- Pilot Readiness

Goal: Every item on the production launch checklist is checked, and 2-3 real clients are ready to go live.

Shivashree	Nishkala	Nihal
Shared -- final checklist review and pilot onboarding.	Shared -- final checklist review and pilot onboarding.	Shared -- final checklist review and pilot onboarding.

Expected output

- Production Launch Checklist (Team Execution Plan §P) fully checked; 2-3 pilot clients onboarded with agents configured and knowledge bases uploaded; pilot success metrics dashboard live

Dependencies

- Every prior checkpoint genuinely green -- this week is verification and onboarding, not a place to finish unfinished work from earlier weeks

Risks

- The temptation to treat this week as buffer for slipped earlier work is real -- if that's what's happening, it should be visible and explicit, not quietly absorbed

Demo checklist

- Walk the full checklist live with the team; place a real call to a real pilot client's configured agent

Acceptance criteria

- All P0 tickets across the Team Execution Plan closed; launch checklist complete; pilot clients live